Byte-Pair Encoding (BPE)

- Tokens are obtained using a process called Byte-Pair Encoding
- Byte-Pair Encoding (BPE) is a **data compression** and text-processing algorithm used by Large Language Models (LLMs) to translate **words into numerical tokens**

Byte-Pair Encoding (BPE)

How tokenizers learn to merge characters into subwords

① Start: Corpus + Base Vocab

"hug"	×10
"pug"	×5
"pun"	×12
"bun"	×4
"hugs"	×5

Base vocab:

`[ "b", "g", "h", "n", "p", "s", "u" ]`

→

② Learn Merge Rules (repeat until vocab full)

Step 1

Most frequent pair:

`("u", "g")` → **"ug"** *+ "ug" added to vocab*

freq 20 — most common

Corpus: `("h""ug", 10), ("p""ug", 5), ("h""ug""s", 5)`

Step 2

Most frequent pair:

`("u", "n")` → **"un"** *+ "un" added to vocab*

freq 16

Corpus: `("p""un", 12), ("b""un", 4)`

Step 3

Most frequent pair:

`("h", "ug")` → **"hug"** *+ "hug" added to vocab*

freq 15

③ Tokenize New Word → apply merge rules in order

"bug" → split → "b" "ug" *← "ug" matched merge rule; "b" stays as char*

Source: Hugging Face LLM Course — Chapter 6: Byte-Pair Encoding Tokenization

Extending the Vocabulary: New Words

Adding mug, smug, chug, thug — applying the same 3 merge rules

Tokenizing New Words

mug → m | ug *"m" + "ug"*

smug → s | m | ug *"s" + "m" + "ug"*

chug → c | hug *"c" + "hug"*

thug → t | hug *"t" + "hug"*

Merge rules applied (in order):

Rule 1 `(u, g) → "ug"` Rule 2 `(u, n) → "un"` Rule 3 `(h, ug) → "hug"`

Final Vocabulary + Token IDs

ID	Token	Type
0	"b"	base char
1	"c"	base char
2	"g"	base char
3	"h"	base char
4	"m"	base char
5	"n"	base char
6	"p"	base char
7	"s"	base char
8	"t"	base char
9	"u"	base char
10	"ug"	*merge rule 1*
11	"un"	*merge rule 2*
12	"hug"	*merge rule 3*

Source: Hugging Face LLM Course — Chapter 6: Byte-Pair Encoding Tokenization

LLM Fundamentals

Why BPE?

Tokens are obtained using a process called Byte-Pair Encoding. This process is necessary because:

- Computers only do math. Words are converted into numbers so the heavy math to understand and generate language can be performed
- BPE creates a fixed size dictionary. If it tries to remember every single word, slang, and typo, the list would be too big
- Numbers are memory and space efficient

LLM Fundamentals

BPE → Embeddings

- Each token is converted into a vector representation called an **embedding** and is also assigned a **token ID**
- Each token is represented by 768 - 4000 dimensional vector in giant lookup table called **embedding layer**
- The size of embedding vector depends upon the model
- Token ID serves as the
    - **"The"** (Token 101) ──▶ Looks up Row 101 ──▶ `[ 0.12, -0.45, 0.89 ]`
    - **"cat"** (Token 102) ──▶ Looks up Row 102 ──▶ `[ 0.84, -0.21, 0.95 ]`
    - **"sat"** (Token 103) ──▶ Looks up Row 103 ──▶ `[-0.33, 0.76, -0.11 ]`